

The Normal Probability Plot

We have already mentioned that when the irreducible errors can be assumed to be iid normal, then there are additional appealing results that can be stated:

1. The least squares estimator can be justified as being the maximum likelihood estimators.
2. The least squares estimators are exactly normal.
3. The least squares estimators, when appropriately scaled, have a t -distribution, which can be used to construct confidence intervals and hypothesis tests.

Overall, however, the value of this assumption is not that great. First, least squares can be motivated even if it does not lead to the MLE. Second, there are versions of the central limit theorem that state that, assuming the errors are independent (and some regularity conditions), the least squares estimator will be approximately normal for “large” n .

Still, it can be useful to assess the normality of the residuals, as a means of assessing the normality of the irreducible error. The standard tool for doing this is the **normal probability plot**, which is a particular version of a **quantile-quantile plot**, or **QQ plot**.

In a QQ plot, one compares the sorted sample values versus the corresponding percentiles of the distribution being tested. More specifically, the i^{th} order statistic is plotted versus the $100(i/(n+1))$ percentile of the distribution, for $i = 1, 2, \dots, n$. If the sample was drawn from that distribution, then the plot should show (approximately) a 45-degree line.

This may require the estimation of parameters of the distribution; if so, then use the best estimates available (e.g., MLE). Fortunately, in the normal case, this is unnecessary because a linear transformation of any normal is also normal. Hence, comparison with the standard normal is sufficient.

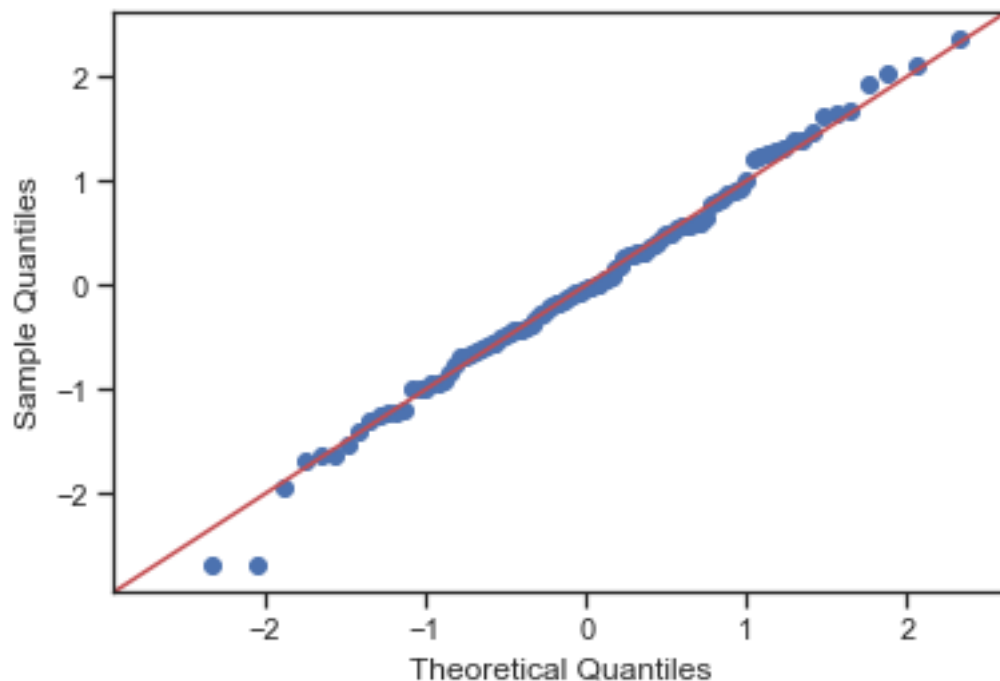
In Python, this plot can be constructed using `qqplot()` found in `statsmodels`. Note the use of the `Fit` argument to force Python to compare with the best fitting distribution within the specified family. (One can obtain fitted parameters using the `fit()` method.)

```
In [25]: from statsmodels.graphics.gofplots import qqplot
import scipy.stats.distributions as spdists

x = spdists.t.rvs(30, size=100)
theodist = spdists.norm

qqplot(x, dist=theodist, line="45", fit=True)
plt.show()
```

$\swarrow$ df $\swarrow$ n



Exercise: Use the above code to try different examples and explore the use of this tool.

Exercise: Explain why checking the normal probability plot of the residuals may be a good idea.

① If sample size is small, then normality assumption is needed to do inference on β_1 using the classical results

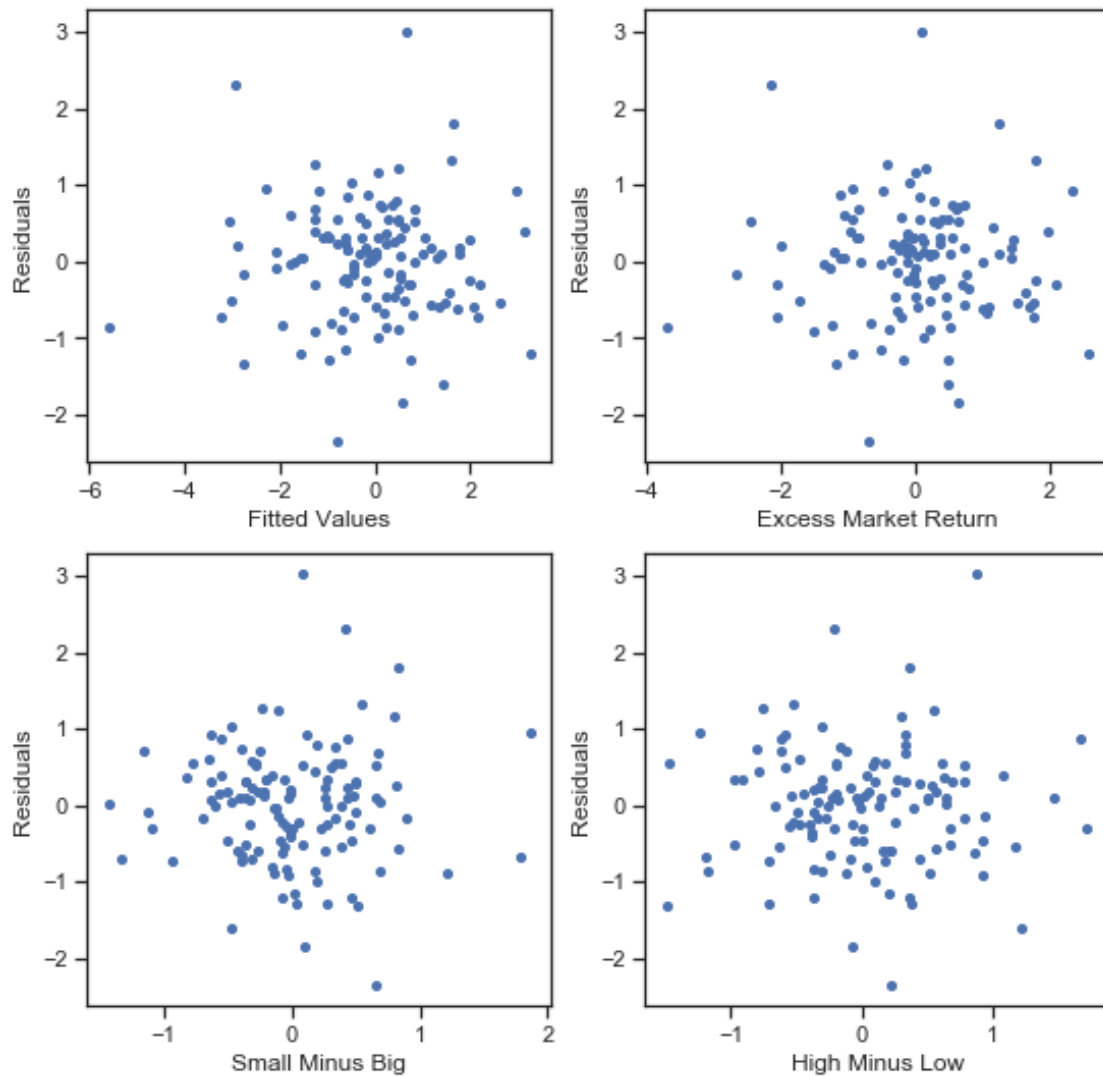
② If you want to use the MLE, then checking normality can justify least squares

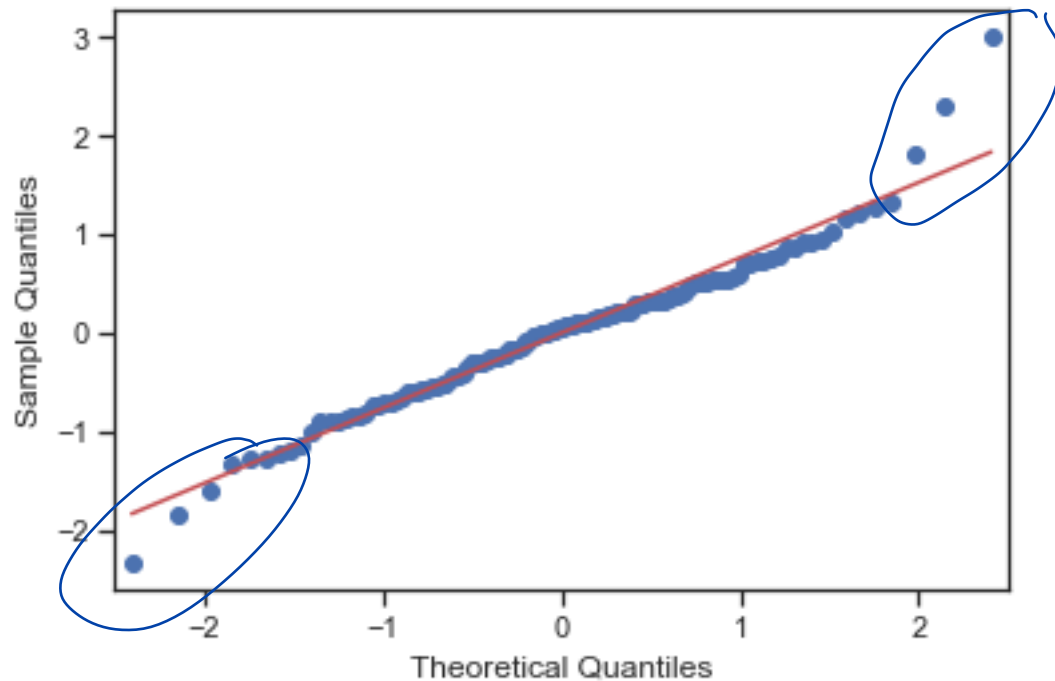
③ Evidence of heavy tails in the normal prob plot could suggest that a procedure other than least squares is worthwhile.
Large errors lead to influential observations.
Might consider "robust" regression procedure.

④ Helps identifying outliers, points in scatter plot which

Let's now return to our PNC example. The figure below shows three diagnostic plots:

1. Plot of residuals versus fitted values.
2. Plot of residuals versus the predictors.
3. Normal probability plot of the residuals.





Exercise: Comment on the quality of the fit.

Plots of residuals versus fitted values and versus predictors show no evidence of lack of fit.

Normal prob. plot suggests ϵ_i have heavier tails than normal.

No evidence of outliers

Linear Regression in Matrix Notation

First, recall our simple linear regression model:

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i, \quad i = 1, 2, \dots, n$$

We can rewrite this model in matrix notation by letting

$$\underline{\mathbf{Y}} = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix} \quad \underline{\mathbf{X}} = \begin{bmatrix} 1 & X_1 \\ 1 & X_2 \\ \vdots & \vdots \\ 1 & X_n \end{bmatrix} \quad \underline{\boldsymbol{\beta}} = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} \quad \underline{\boldsymbol{\epsilon}} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

and then noting that

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon} \quad Y_1 = 1 \cdot \beta_0 + X_1 \cdot \beta_1 + \epsilon_1$$

When moving to multiple regression, the main change we must make is to add columns to the matrix $\mathbf{X}$, commonly called the **design matrix**, and add entries to $\boldsymbol{\beta}$:

$$\mathbf{X} = \begin{bmatrix} 1 & X_{11} & X_{12} & \cdots & X_{1,p} \\ 1 & X_{21} & X_{22} & \cdots & X_{2,p} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & X_{n1} & X_{n2} & \cdots & X_{n,p} \end{bmatrix} \quad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}$$

One can show that the least squares estimators (now a vector) are

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}.$$

and can now define the vector of fitted values and residuals as

$$\hat{\mathbf{Y}} = \begin{bmatrix} \hat{Y}_1 \\ \hat{Y}_2 \\ \vdots \\ \hat{Y}_n \end{bmatrix} \quad \text{and} \quad \hat{\boldsymbol{\epsilon}} = \begin{bmatrix} \hat{\epsilon}_1 \\ \hat{\epsilon}_2 \\ \vdots \\ \hat{\epsilon}_n \end{bmatrix}$$

and it follows that

$$\begin{aligned} \hat{\mathbf{Y}} &= \mathbf{X} \hat{\beta} \quad \text{and} \quad \hat{\boldsymbol{\epsilon}} = \mathbf{Y} - \hat{\mathbf{Y}}. \\ &= \underbrace{\mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T}_{\sim} \mathbf{Y}_{\sim} \end{aligned}$$

The Hat Matrix

We note that we could write

$$\hat{\mathbf{Y}} = \mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y} = \mathbf{H}\mathbf{Y}$$

where

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$$

is called the **hat matrix** (because it puts a “hat” on $\mathbf{Y}$).

The diagonal entries of $\mathbf{H}$ are referred to as the **leverages**.

Comment: Mathematically, $\mathbf{H}$ is a **projection matrix**, as it “projects” the vector of observed responses $\mathbf{Y}$ onto the space of all vectors which are linear combinations of the columns of $\mathbf{X}$. In fact, we note that $\mathbf{H}$ is symmetric and **idempotent**, meaning that $\mathbf{H}\mathbf{H} = \mathbf{H}$.

Exercise: Show that $\mathbf{I} - \mathbf{H}$ is also symmetric and idempotent.

$$(\mathbf{I} - \mathbf{H})^T = \mathbf{I}^T - \mathbf{H}^T = \mathbf{I} - \mathbf{H}$$

$$\begin{aligned}(\mathbf{I} - \mathbf{H})(\mathbf{I} - \mathbf{H}) &= \mathbf{I}\mathbf{I} - \mathbf{I}\mathbf{H} - \mathbf{H}\mathbf{I} + \mathbf{H}\mathbf{H} \\&= \mathbf{I} - \mathbf{H} - \mathbf{H} + \mathbf{H} \\&= \mathbf{I} - \mathbf{H}\end{aligned}$$

$$(\mathbf{I} - \mathbf{H})\mathbf{Y} = \mathbf{Y} - \mathbf{H}\mathbf{Y} = \mathbf{Y} - \hat{\mathbf{Y}} = \hat{\boldsymbol{\epsilon}}$$

$(\mathbf{I} - \mathbf{H})$ projects onto the space orthogonal to the columns of $\mathbf{X}$

Additional Results

A range of important results can be proven:

1. The least squares estimator for β is $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$.
2. The variance of $\hat{\beta}$ is $\sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}$.
3. $\hat{\mathbf{Y}} = \mathbf{X} \hat{\beta} = \mathbf{H} \mathbf{Y}$, where the (symmetric, idempotent) hat matrix $\mathbf{H}$ is defined as above.
4. $\hat{\epsilon} = \mathbf{Y} - \hat{\mathbf{Y}} = (\mathbf{I} - \mathbf{H}) \mathbf{Y}$.
5. If ϵ is assumed normal with mean zero and variance $\sigma^2 \mathbf{I}$, the following hold:
 - $\hat{\beta}$ is the maximum likelihood estimator.
 - $\mathbf{Y}$ is multivariate normal with mean $\mathbf{X}\beta$ and covariance $\sigma^2 \mathbf{I}$.
 - $\hat{\beta}$ is multivariate normal with mean β and covariance $\sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}$.
 - $\hat{\mathbf{Y}}$ is multivariate normal with mean $\mathbf{X}\beta$ and covariance $\sigma^2 \mathbf{H}$.
 - $\hat{\epsilon}$ is multivariate normal with mean zero and covar. $\sigma^2 (\mathbf{I} - \mathbf{H})$.

$$\begin{aligned}
 V(\hat{\beta}) &= V\left(\underbrace{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T}_{\sim} \underbrace{\mathbf{Y}}_{\sim}\right) = \underbrace{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T}_{\sim} V(\mathbf{Y}) \underbrace{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T}_{\sim}^T \\
 &= (\mathbf{X}^T \mathbf{X})^{-1} \sigma^2 \mathbf{I}
 \end{aligned}$$

$$\hat{\epsilon} = (\mathbf{I} - \mathbf{H}) \mathbf{Y}$$

$$\begin{aligned}
 V(\hat{\epsilon}) &= (\mathbf{I} - \mathbf{H}) V(\mathbf{Y}) (\mathbf{I} - \mathbf{H}) \\
 &= (\mathbf{I} - \mathbf{H}) \sigma^2
 \end{aligned}$$

Degrees of Freedom

One key thing that does change when moving to multiple regression is the number of **degrees of freedom** in the residual vector.

Recall that we had seen previously that with simple linear regression, it held that

$$\sum_i \hat{\epsilon}_i = 0 \quad \text{and} \quad \sum_i \hat{\epsilon}_i X_i = 0.$$

This result is one way of seeing why the residuals only have $n-2$ degrees of freedom in the case of simple linear regression: There are two constraints placed on the vector of residuals.

But, these facts could be written more compactly using matrix notation: $\mathbf{X}^T \hat{\epsilon} = \mathbf{0}$, where $\mathbf{0}$ is a vector consisting entirely of zeros.

In fact, this result extends to multiple regression: The vector of residuals $\hat{\epsilon}$ is orthogonal to **every column** of $\mathbf{X}$, i.e., $\mathbf{X}^T \hat{\epsilon} = \mathbf{0}$.

We now can state that there are $n - (p + 1)$ degrees of freedom in the residuals, and hence need to update some of our previous results. (You will note that simple linear regression corresponds to the case where $p = 1$.)

In particular:

1. The unbiased estimator of σ^2 is

$$\hat{\sigma}^2 = \text{RSS} / \underline{(n - p - 1)}$$

2. When the errors of i.i.d. normal, the statistic

$$\frac{\hat{\beta}_i - \beta_i}{\widehat{\text{SE}}(\hat{\beta}_i)} \quad \leftarrow$$

has the t -distribution with $n - p - 1$ degrees of freedom. Hence, a $100(1 - \alpha)\%$ confidence interval for β_i is formed as

$$\hat{\beta}_i \pm \underline{t_{\alpha/2, n-p-1}} \widehat{\text{SE}}(\hat{\beta}_i)$$

and hypothesis tests concerning β_i should compare the test statistic (the “t value”) with the t -distribution with $n - p - 1$ degrees of freedom.

3. For large n , we can appeal to the central limit theorem and construct an approximate $100(1 - \alpha)\%$ confidence interval for β_i using

$$\hat{\beta}_i \pm z_{\alpha/2} \widehat{\text{SE}}(\hat{\beta}_i)$$

Basic Variable Selection

The process of **variable selection** in regression consists of making a decision among all of the possible subsets of covariates to find that model with the best ability to **generalize** to the population from which the training sample was drawn.

Various ~~criteria~~^{on} have been proposed for comparing models.

A proven, general strategy for predictor selection is to compare the values of the **Akaike Information Criterion (AIC)** across different models. AIC is defined as

$$\text{AIC} = n \log(\text{RSS}) + 2p$$

$\text{RSS} = \sum \hat{\epsilon}_i^2$

where p is the number of β parameters in the model.

Exercise: Justify the form for AIC. How does it balance model fit with model complexity?

As increase # of β parameters, RSS will decrease, so $n \log(\text{RSS})$ will decrease

The " $2p$ " penalizes for overly complex models

Is the addition of this new β parameter important enough? Is it reducing RSS enough to justify its inclusion?

General Form $\text{AIC} = -2 \log \text{likelihood} + 2p$

Technical Comment:

The actual value of AIC is not important, it is only the ordering of the models that it provides which is important.

Different texts and software packages use different definitions of AIC in the context of linear regression. In all cases, these expressions should be a monotonically increasing function of the definition given above. Hence, these definitions do not differ in ways that are of practical importance, i.e., the ordering of models that results will be the same.

The value of AIC reported by Python for the model fit above is as follows:

```
In [9]: ff3modPNC.aic
```

```
Out[9]: 292.7014744341992
```

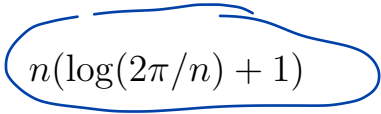
while the value using the formula calculated above is

```
In [10]: import numpy as np
```

```
ff3modPNC.nobs*np.log(sum(ff3modPNC.resid**2)) + 2*4
```

```
Out[10]: 535.5412709766572
```

These differ by the factor


$$n(\log(2\pi/n) + 1)$$

The reason for this difference will be discussed later when we use AIC in general situations.